6 Working with configuration properties

This chapter covers

- Fine-tuning autoconfigured beans
- Applying configuration properties to application components
- Working with Spring profiles

Do you remember when the iPhone first came out? A small slab of metal and glass hardly fit the description of what the world had come to recognize as a phone. And yet, it pioneered the modern smartphone era, changing everything about how we communicate. Although touch phones are in many ways easier and more powerful than their predecessor, the flip phone, when the iPhone was first announced, it was hard to imagine how a device with a single button could be used to place calls.

In some ways, Spring Boot autoconfiguration is like this. Autoconfiguration greatly simplifies Spring application development. But after a decade of setting property values in Spring XML configuration and calling setter methods on bean instances, it's not immediately apparent how to set properties on beans for which there's no explicit configuration.

Fortunately, Spring Boot provides a way to set property values on application components with configuration properties. Configuration properties are nothing more than properties on `@ConfigurationProperties`-annotated beans in the Spring application context. Spring will inject values from one of several property sources—including JVM system properties, command-line arguments, and environment variables—into the bean properties. We'll see how to use `@ConfigurationProperties` on our own beans in section 6.2. But Spring Boot itself provides several `@ConfigurationProperties`-annotated beans that we'll configure first.

In this chapter, you're going to take a step back from implementing new features in the Taco Cloud application to explore configuration properties. What you take away will no doubt prove useful as you move forward in the chapters that follow. We'll start by seeing how to employ configuration properties to fine-tune what Spring Boot automatically configures.

6.1 Fine-tuning autoconfiguration

Before we dive in too deeply with configuration properties, it's important to establish the following different (but related) kinds of configurations in Spring:

- *Bean wiring*—Configuration that declares application components to be created as beans in the Spring application context and how they should be injected into each other
- *Property injection*—Configuration that sets values on beans in the Spring application context

In Spring's XML and Java configuration, these two types of configurations are often declared explicitly in the same place. In Java configuration, a `@Bean`-annotated method is likely to both instantiate a bean and then set values to its properties. For example, consider the following `@Bean` method that declares a `DataSource` for an embedded H2 database:

```
@Bean
public DataSource dataSource() {
  return new EmbeddedDatabaseBuilder()
      .setType(H2)
      .addScript("taco_schema.sql")
      .addScripts("user_data.sql", "ingredient_data.sql")
      .build();
}
```

Here the `addScript()` and `addScripts()` methods set some `String` properties with the name of SQL scripts that should be applied to the database once the data source is ready. Whereas this is how you might configure a `DataSource` bean if you aren't using Spring Boot, autoconfiguration makes this method completely unnecessary.

If the H2 dependency is available in the runtime classpath, then Spring Boot automatically creates in the Spring application context an appropriate `DataSource` bean, which applies the SQL scripts schema.sql and data.sql.

But what if you want to name the SQL scripts something else? Or what if you need to specify more than two SQL scripts? That's where configuration properties come in. But before you can start using configuration properties, you need to understand where those properties come from.

6.1.1 Understanding Spring's environment abstraction

The Spring environment abstraction is a one-stop shop for any configurable property. It abstracts the origins of properties so that beans needing those properties can consume them from Spring itself. The Spring environment pulls from several property sources, including the following:

- JVM system properties
- Operating system environment variables
- Command-line arguments
- Application property configuration files

It then aggregates those properties into a single source from which Spring beans can be injected. Figure 6.1 illustrates how properties from property sources flow through the Spring environment abstraction to Spring beans.

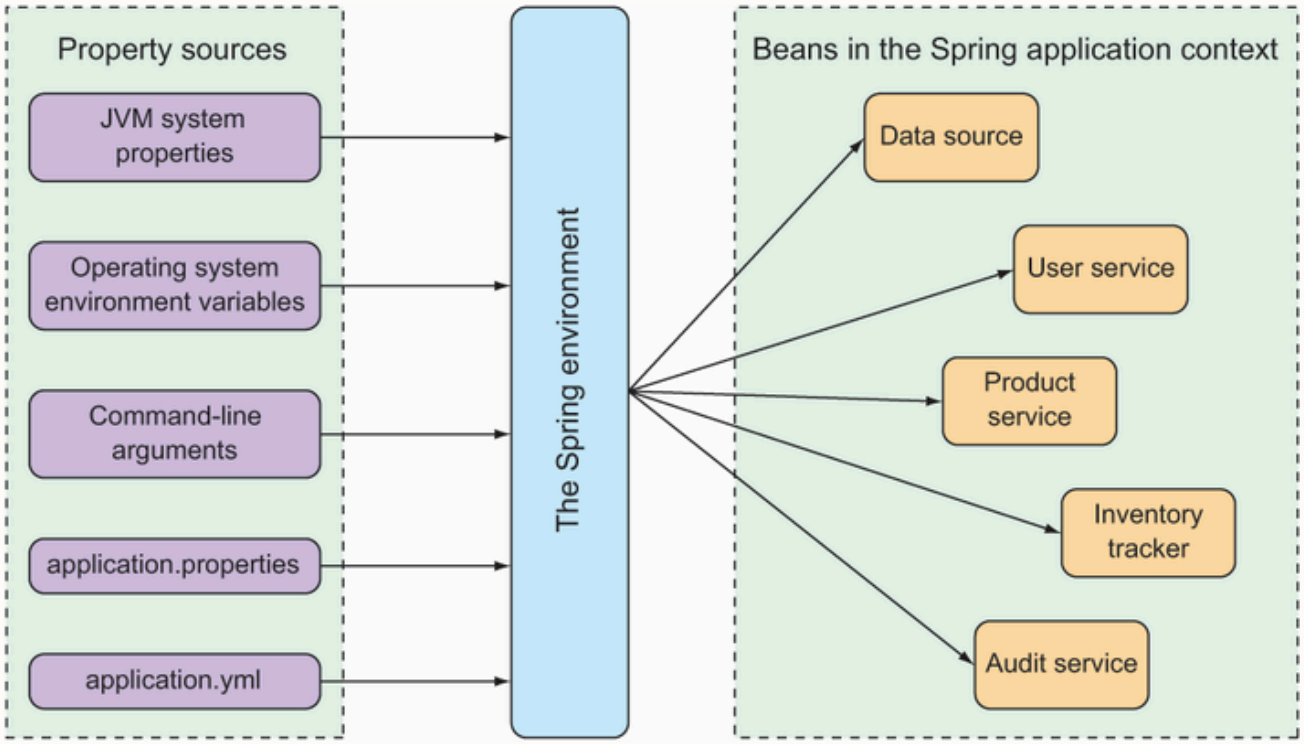


Figure 6.1 The Spring environment pulls properties from property sources and makes them available to beans in the application context.

The beans that are automatically configured by Spring Boot are all configurable by properties drawn from the Spring environment. As a simple example, suppose that you would like the application's underlying servlet container to listen for requests on some port other than the default port of 8080. To do that, specify a different port by setting the `server.port` property in src/main/resources/application.properties like this:

```
server.port=9090
```

Personally, I prefer using YAML when setting configuration properties. Therefore, instead of using application.properties, I might set the `server.port` value in src/main/resources/application.yml like this:

```
server:
  port: 9090
```

If you'd prefer to configure that property externally, you could also specify the port when starting the application using a command-line argument as follows:

```
$ java -jar tacocloud-0.0.5-SNAPSHOT.jar --server.port=9090
```

If you want the application to always start on a specific port, you could set it one time as an operating system environment variable, as shown next:

```
$ export SERVER_PORT=9090
```

Notice that when setting properties as environment variables, the naming style is slightly different to accommodate restrictions placed on environment variable names by the operating system. That's OK. Spring is able to sort it out and interpret `SERVER_PORT` as `server.port` with no problems.

As I said, we have several ways of setting configuration properties. In fact, you could use one of several hundred configuration properties to tweak and adjust how Spring beans behave. You've already seen a few: `server.port` in this chapter, as well as `spring.datasource.name` and `spring.thymeleaf.cache` in earlier chapters.

It's impossible to examine all of the available configuration properties in this chapter. Even so, let's take a look at a few of the most useful configuration properties you might commonly encounter. We'll start with a few properties that let you tweak the autoconfigured data source.

1.2 Configuring a data source

At this point, the Taco Cloud application is still unfinished, but you'll have several more chapters to take care of that before you're ready to deploy the application. As such, the embedded H2 database you're using as a data source is perfect for your needs—for now.

But once you take the application into production, you'll probably want to consider a more permanent database solution.

Although you could explicitly configure your own `DataSource` bean, that's usually unnecessary. Instead, it's simpler to configure the URL and credentials for your database via configuration properties. For example, if you were to start using a MySQL database, you might add the following configuration properties to application.yml:

```
spring:
  datasource:
    url: jdbc:mysql:/ /localhost/tacocloud
    username: tacouser
    password: tacopassword
```

Although you'll need to add the appropriate JDBC driver to the build, you won't usually need to specify the JDBC driver class—Spring Boot can figure it out from the structure of the database URL. But if there's a problem, you can try setting the `spring.datasource.driver-class-name` property like so:

```
spring:
  datasource:
    url: jdbc:mysql:/ /localhost/tacocloud
    username: tacouser
    password: tacopassword
    driver-class-name: com.mysql.jdbc.Driver
```

Spring Boot uses this connection data when autoconfiguring the `DataSource` bean. The `DataSource` bean will be pooled using the HikariCP connection pool if it's available on the classpath. If not, Spring Boot looks for and uses one of the following other connection pool implementations on the classpath:

- Tomcat JDBC Connection Pool
- Apache Commons DBCP2

Although these are the only connection pool options available through autoconfiguration, you're always welcome to explicitly configure a `DataSource` bean to use whatever connection pool implementation you'd like.

Earlier in this chapter, we suggested that there might be a way to specify the database initialization scripts to run when the application starts. In that case, the

`spring.sql.init.schema-locations` property proves useful, as shown here:

```
spring:
sql:
init:
schema-locations:
- order-schema.sql
- ingredient-schema.sql
- taco-schema.sql
- user-schema.sql
- ingredients.sql
```

Maybe explicit data source configuration isn't your style. Instead, perhaps you'd prefer to configure your data source in the Java Naming and Directory Interface (JNDI) (**http://mng.bz/MvEo**) and have Spring look it up from there. In that case, set up your data source by configuring `spring.datasource.jndi-name` as follows:

```
spring:
  datasource:
    jndi-name: java:/comp/env/jdbc/tacoCloudDS
```

If you set the `spring.datasource.jndi-name` property, the other data source connection properties (if set) are ignored.

1.3 Configuring the embedded server

You've already seen how to set the servlet container's port by setting `server.port`. What I didn't show you is what happens if `server.port` is set to `0`, as shown here:

```
server:
  port: 0
```

Although you're explicitly setting `server.port` to `0`, the server won't start on port 0. Instead, it'll start on a randomly chosen available port. This is useful when running automated integration tests to ensure that any concurrently running tests don't clash on a hardcoded port number.

But there's more to the underlying server than just a port. One of the most common things you'll need to do with the underlying container is to set it up to handle HTTPS re-

quests. To do that, the first thing you must do is create a keystore using the JDK's `keytool` command-line utility, as shown next:

```
$ keytool -keystore mykeys.jks -genkey -alias tomcat -keyalg RSA
```

You'll be asked several questions about your name and organization, most of which are irrelevant. But when asked for a password, remember what you choose. For the sake of this example, I chose letmein as the password.

Next, you'll need to set a few properties to enable HTTPS in the embedded server. You could specify them all on the command line, but that would be terribly inconvenient. Instead, you'll probably set them in the application.properties or application.yml file. In application.yml, the properties might look like this:

```
server:
  port: 8443
  ssl:
    key-store: file:/ / /path/to/mykeys.jks
    key-store-password: letmein
    key-password: letmein
```

Here the) `server.port` property is set to `8443`, a common choice for development HTTPS servers. The `server.ssl.key-store` property should be set to the path where the keystore file is created. Here it's shown with a `file:/ /` URL to load it from the filesystem, but if you package it within the application JAR file, you'll use a `classpath:` URL to reference it. And both the `server.ssl.key-store-password` and `server.ssl.key-password` properties are set to the password that was given when creating the keystore.

With these properties in place, your application should be listening for HTTPS requests on port 8443. Depending on which browser you're using, you may encounter a warning about the server not being able to verify its identity. This is nothing to worry about when serving from localhost during development.

1.1.4 Configuring logging

Most applications provide some form of logging. And even if your application doesn't log anything directly, the libraries that your application uses will certainly log their activity.

By default, Spring Boot configures logging via Logback (**http://logback.qos.ch**) to write to the console at an `INFO` level. You've probably already seen plenty of `INFO`-level en-

tries in the application logs as you've run the application and other examples. But as a reminder, here's a logging sample showing the default log format (wrapped to fit within the page margins):

```
2021-07-29 17:24:24.187 INFO 52240 --- [nio-8080-exec-1]
↪ com.example.demo.Hello              Here's a log entry.
2021-07-29 17:24:24.187 INFO 52240 --- [nio-8080-exec-1]
↪ com.example.demo.Hello              Here's another log entry.
2021-07-29 17:24:24.187 INFO 52240 --- [nio-8080-exec-1]
↪ com.example.demo.Hello              And here's one more.
```

For full control over the logging configuration, you can create a logback.xml file at the root of the classpath (in src/main/resources). Here's an example of a simple logback.xml file you might use:

```
<configuration>
 <appender name="STDOUT" class="ch.qos.logback.core.ConsoleAppender">
  <encoder>
   <pattern>
    %d{HH:mm:ss.SSS} [%thread] %-5level %logger{36} - %msg%n
   </pattern>
  </encoder>
 </appender>
 <logger name="root" level="INFO"/>
 <root level="INFO">
  <appender-ref ref="STDOUT" />
 </root>
</configuration>
```

With this new configuration, the same sample log entries from earlier might look like this (wrapped to fit in the page margins):

```
17:25:09.088 [http-nio-8080-exec-1] INFO  com.example.demo.Hello -
                                    Here's a log entry.
17:25:09.088 [http-nio-8080-exec-1] INFO  com.example.demo.Hello -
                                    Here's another log entry.
17:25:09.088 [http-nio-8080-exec-1] INFO  com.example.demo.Hello -
                                    And here's one more.
```

Aside from the pattern used for logging, this Logback configuration is more or less equivalent to the default you'll get if you have no logback.xml file. But by editing

logback.xml, you can gain full control over your application's log files.

NOTE The specifics of what can go into logback.xml are outside the scope of this book. Refer to Logback's documentation for more information.

The most common changes you'll make to a logging configuration are to change the logging levels and perhaps to specify a file where the logs should be written. With Spring Boot configuration properties, you can make those changes without having to create a logback.xml file.

To set the logging levels, you create properties that are prefixed with `logging.level`, followed by the name of the logger for which you want to set the logging level. For instance, suppose you'd like to set the root logging level to `WARN`, but log Spring Security logs at a `DEBUG` level. The following entries in application.yml will take care of that for you:

```
logging:
  level:
    root: WARN
    org:
      springframework:
        security: DEBUG
```

Optionally, you can collapse the Spring Security package name to a single line for easier reading as follows:

```
logging:
  level:
    root: WARN
    org.springframework.security: DEBUG
```

Now suppose that you want to write the log entries to the file TacoCloud.log at /tmp/logs/. The `logging.file.path` and `logging.file.name` properties can help achieve that, as shown next:

```
logging:
  file:
    name: /tmp/logs/TacoCloud.log
  level:
    root: WARN
```

```
    org:
      springframework:
        security: DEBUG
```

Assuming that the application has write permissions to /tmp/logs/, the log entries will be written to /tmp/logs/TacoCloud.log. By default, the log files rotate once they reach 10 MB in size.

1.5 Using special property values

When setting properties, you aren't limited to declaring their values as hardcoded `String` and numeric values. Instead, you can derive their values from other configuration properties.

For example, suppose (for whatever reason) you want to set a property named `greeting.welcome` to echo the value of another property named `spring.application.name`. To achieve this, you could use the `${}` placeholder markers when setting `greeting.welcome` as follows:

```
greeting:
  welcome: ${spring.application.name}
```

You can even embed that placeholder amid other text, as shown here:

```
greeting:
  welcome: You are using ${spring.application.name}.
```

As you've seen, configuring Spring's own components with configuration properties makes it easy to inject values into those components' properties and to fine-tune auto-configuration. Configuration properties aren't exclusive to the beans that Spring creates. With a small amount of effort, you can take advantage of configuration properties in your own beans. Let's see how.

6.2 Creating your own configuration properties

As I mentioned earlier, configuration properties are nothing more than properties of beans that have been designated to accept configurations from Spring's environment abstraction. What I didn't mention is how those beans are designated to consume those configurations.

To support property injection of configuration properties, Spring Boot provides the `@ConfigurationProperties` annotation. When placed on any Spring bean, it specifies that the properties of that bean can be injected from properties in the Spring environment.

To demonstrate how `@ConfigurationProperties` works, suppose that you've added the following method to `OrderController` to list the authenticated user's past orders:

```
@GetMapping
public String ordersForUser(
    @AuthenticationPrincipal User user, Model model) {

  model.addAttribute("orders",
      orderRepo.findByUserOrderByPlacedAtDesc(user));

  return "orderList";
}
```

Along with that, you've also added the next necessary `findByUserOrderByPlacedAt-Desc()` method to `OrderRepository`:

```
List<Order> findByUserOrderByPlacedAtDesc(User user);
```

Notice that this repository method is named with a clause of `OrderByPlacedAtDesc`. The `OrderBy` portion specifies a property by which the results will be ordered—in this case, the `placedAt` property. The `Desc` at the end causes the ordering to be in descending order. Therefore, the list of orders returned will be sorted from most recent to least recent.

As written, this controller method may be useful after the user has placed a handful of orders, but it could become a bit unwieldy for the most avid of taco connoisseurs. A few orders displayed in the browser are useful; a never-ending list of hundreds of orders is just noise. Let's say that you want to limit the number of orders displayed to the most recent 20 orders. You can change `ordersForUser()` as follows:

```
@GetMapping
public String ordersForUser(
    @AuthenticationPrincipal User user, Model model) {

  Pageable pageable = PageRequest.of(0, 20);
  model.addAttribute("orders",
```

```
        orderRepo.findByUserOrderByPlacedAtDesc(user, pageable));

    return "orderList";
  }
```

along with the corresponding changes to `OrderRepository`, shown next:

```
List<TacoOrder> findByUserOrderByPlacedAtDesc(
        User user, Pageable pageable);
```

Here you've changed the signature of the `findByUserOrderByPlacedAtDesc()` method to accept a `Pageable` as a parameter. `Pageable` is Spring Data's way of selecting some subset of the results by a page number and page size. In the `ordersForUser()` controller method, you constructed a `PageRequest` object that implemented `Pageable` to request the first page (page zero) with a page size of 20 to get up to 20 of the most recently placed orders for the user.

Although this works fantastically, it leaves me a bit uneasy that you've hardcoded the page size. What if you later decide that 20 is too many orders to list, and you decide to change it to 10? Because it's hardcoded, you'd have to rebuild and redeploy the application.

Rather than hardcode the page size, you can set it with a custom configuration property. First, you need to add a new property called `pageSize` to `OrderController`, and then annotate `OrderController` with `@ConfigurationProperties` as shown in the next listing.

Listing 6.1 Enabling configuration properties in `OrderController`

```
@Controller
@RequestMapping("/orders")
@SessionAttributes("order")
@ConfigurationProperties(prefix="taco.orders")
public class OrderController {

  private int pageSize = 20;

  public void setPageSize(int pageSize) {
    this.pageSize = pageSize;
  }

  ...
```

```
  @GetMapping
  public String ordersForUser(
      @AuthenticationPrincipal User user, Model model) {

    Pageable pageable = PageRequest.of(0, pageSize);
    model.addAttribute("orders",
        orderRepo.findByUserOrderByPlacedAtDesc(user, pageable));
    return "orderList";
  }

}
```

The most significant change made in listing 6.1 is the addition of the `@ConfigurationProperties` annotation. Its prefix attribute is set to `taco.orders`, which means that when setting the `pageSize` property, you need to use a configuration property named `taco.orders.pageSize`.

The new `pageSize` property defaults to `20`, but you can easily change it to any value you want by setting a `taco.orders.pageSize` property. For example, you could set this property in application.yml like this:

```
taco:
  orders:
    pageSize: 10
```

Or, if you need to make a quick change while in production, you can do so without having to rebuild and redeploy the application by setting the `taco.orders.pageSize` property as an environment variable as follows:

```
$ export TACO_ORDERS_PAGESIZE=10
```

Any means by which a configuration property can be set can be used to adjust the page size of the recent orders page. Next, we'll look at how to set configuration data in property holders.

2.1 Defining configuration property holders

There's nothing that says `@ConfigurationProperties` must be set on a controller or any other specific kind of bean. `@ConfigurationProperties` are in fact often placed on beans whose sole purpose in the application is to be holders of configuration data. This keeps

configuration-specific details out of the controllers and other application classes. It also makes it easy to share common configuration properties among several beans that may make use of that information.

In the case of the `pageSize` property in `OrderController`, you could extract it to a separate class. The following listing uses the `OrderProps` class in such a way.

Listing 6.2 Extracting `pageSize` to a holder class

```
package tacos.web;
import org.springframework.boot.context.properties.
                                ConfigurationProperties;
import org.springframework.stereotype.Component;
import lombok.Data;

@Component
@ConfigurationProperties(prefix="taco.orders")
@Data
public class OrderProps {

  private int pageSize = 20;

}
```

As you did with `OrderController`, the `pageSize` property defaults to `20`, and `OrderProps` is annotated with `@ConfigurationProperties` to have a prefix of `taco.orders`. It's also annotated with `@Component` so that Spring component scanning will automatically discover it and create it as a bean in the Spring application context. This is important, because the next step is to inject the `OrderProps` bean into `OrderController`.

There's nothing particularly special about configuration property holders. They're beans that have their properties injected from the Spring environment. They can be injected into any other bean that needs those properties. For `OrderController`, this means removing the `pageSize` property from `OrderController` and instead injecting and using the `OrderProps` bean, as shown next:

```
private OrderProps props;

public OrderController(OrderRepository orderRepo,
      OrderProps props) {
  this.orderRepo = orderRepo;
```

```
    this.props = props;
  }

    ...

  @GetMapping
  public String ordersForUser(
      @AuthenticationPrincipal User user, Model model) {

    Pageable pageable = PageRequest.of(0, props.getPageSize());
    model.addAttribute("orders",
        orderRepo.findByUserOrderByPlacedAtDesc(user, pageable));

    return "orderList";
  }
```

Now `OrderController` is no longer responsible for handling its own configuration properties. This keeps the code in `OrderController` slightly neater and allows you to reuse the properties in `OrderProps` in any other bean that may need them. Moreover, you're collecting configuration properties that pertain to orders in one place: the `OrderProps` class. If you need to add, remove, rename, or otherwise change the properties therein, you need to apply those changes only in `OrderProps`. And for testing purposes, it's easy to set configuration properties directly on a test-specific `OrderProps` and give it to the controller prior to the test.

For example, let's pretend that you're using the `pageSize` property in several other beans when you decide it would be best to apply some validation to that property to limit its values to no less than `5` and no more than `25`. Without a holder bean, you'd have to apply validation annotations to `OrderController`, the `pageSize` property, and all other classes using that property. But because you've extracted `pageSize` into `OrderProps`, you only must make the changes to `OrderProps`, as shown here:

```
package tacos.web;
import javax.validation.constraints.Max;
import javax.validation.constraints.Min;

import org.springframework.boot.context.properties.
                                ConfigurationProperties;
import org.springframework.stereotype.Component;
import org.springframework.validation.annotation.Validated;

import lombok.Data;
```

```
@Component
@ConfigurationProperties(prefix="taco.orders")
@Data
@Validated
public class OrderProps {

  @Min(value=5, message="must be between 5 and 25")
  @Max(value=25, message="must be between 5 and 25")
  private int pageSize = 20;

}
```

Although you could as easily apply the `@Validated`, `@Min`, and `@Max` annotations to `OrderController` (and any other beans that can be injected with `OrderProps`), it would just clutter up `OrderController` that much more. With a configuration property holder bean, you've collected configuration property specifics in one place, leaving the classes that need those properties relatively clean.

2.2 Declaring configuration property metadata

Depending on your IDE, you may have noticed that the `taco.orders.pageSize` entry in application.yml (or application.properties) has a warning saying something like Unknown Property 'taco'. This warning appears because there's missing metadata concerning the configuration property you just created. Figure 6.2 shows what this looks like when I hover over the `taco` portion of the property in the Spring Tool Suite.

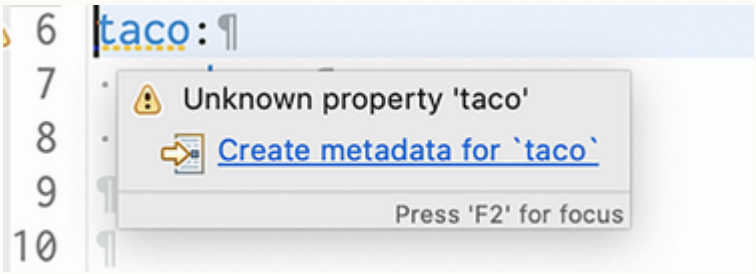


Figure 6.2 A warning for missing configuration property metadata

Configuration property metadata is completely optional and doesn't prevent configuration properties from working. But the metadata can be useful for providing some minimal documentation around the configuration properties, especially in the IDE. For example, when I hover over the `spring.security.user.password` property, I see what's shown in figure 6.3. Although the hover help you get is minimal, it can be enough to help understand what the property is used for and how to use it.

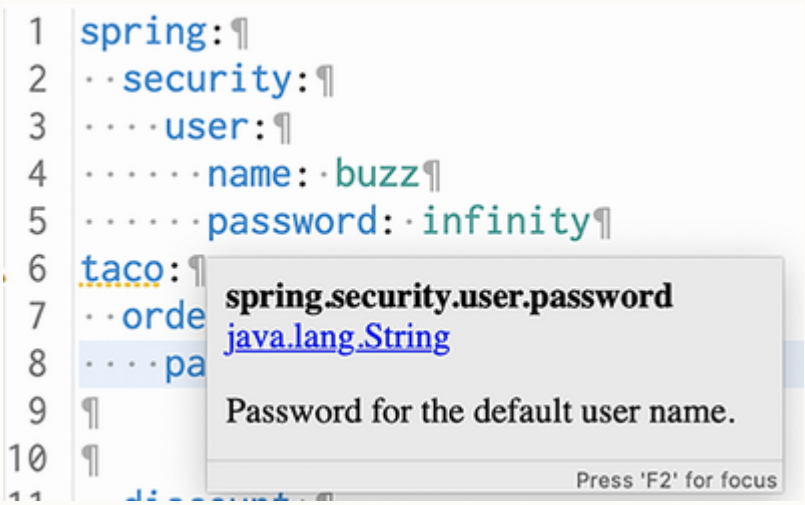


Figure 6.3 Hover documentation for configuration properties in the Spring Tool Suite

To help those who might use the configuration properties that you define—which might even be you—it's generally a good idea to create some metadata around those properties. At least it gets rid of those annoying yellow warnings in the IDE.

To create metadata for your custom configuration properties, you'll need to create a file under the META-INF (e.g., in the project under src/main/resources/META-INF) named additional-spring-configuration-metadata.json.

QUICK-FIXING MISSING METADATA

If you're using the Spring Tool Suite, there's a quick-fix option for creating missing property metadata. Place your cursor on the line with the missing metadata warning and open the quick-fix pop-up with CMD-1 on Mac or Ctrl-1 on Windows and Linux (see figure 6.4).

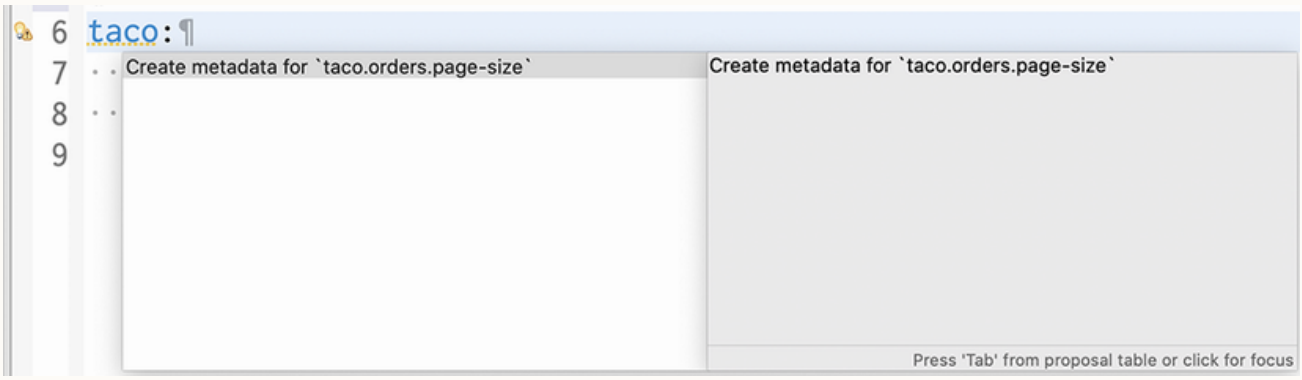


Figure 6.4 Creating configuration property metadata with the quick-fix pop-up in Spring Tool Suite

Then select the "Create Metadata for ..." option to add some metadata for the property. If it doesn't already exist, this quick fix will create a file in META-INF/additional-spring-

configuration-metadata.json and fill it in with some metadata for the `pageSize` property, as shown in the next code:

```
{"properties": [{
  "name": "taco.orders.page-size",
  "type": "java.lang.String",
  "description": "A description for 'taco.orders.page-size'"
}]}
```

Notice that the property name referenced in the metadata is `taco.orders.page-size`, whereas the actual property name in application.yml is `pageSize`. Spring Boot's flexible property naming allows for variations in property names such that `taco.orders .page-size` is equivalent to `taco.orders.pageSize`, so it doesn't matter much which form you use.

The initial metadata written to additional-spring-configuration-metadata.json is a fine start, but you'll probably want to edit it a little. Firstly, the `pageSize` property isn't a `java.lang.String`, so you'll want to change it to `java.lang.Integer`. And the `description` property should be changed to be more descriptive of what `pageSize` is for. The following JSON code sample shows what the metadata might look like after a few edits:

```
{"properties": [{
  "name": "taco.orders.page-size",
  "type": "java.lang.Integer",
  "description": "Sets the maximum number of orders to display in a list."
}]}
```

With that metadata in place, the warnings should be gone. What's more, if you hover over the `taco.orders.pageSize` property, you'll see the description shown in figure 6.5.

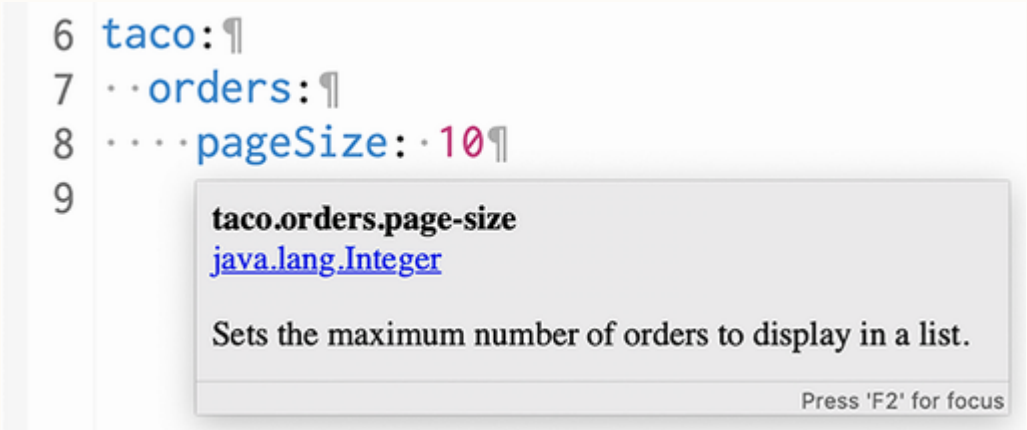


Figure 6.5 Hover help for custom configuration properties

Also, as shown in figure 6.6, you get autocompletion help from the IDE, just like Spring-provided configuration properties.

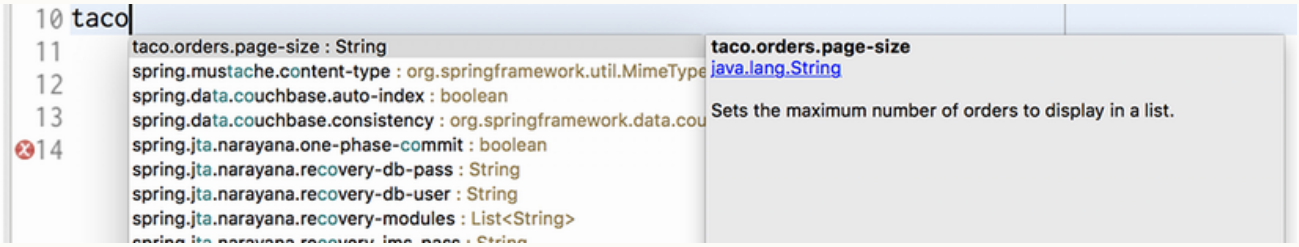


Figure 6.6 Configuration property metadata enables autocompletion of properties.

As you've seen, configuration properties are useful for tweaking both autoconfigured components as well as the details injected into your own application beans. But what if you need to configure different properties for different deployment environments? Let's take a look at how to use Spring profiles to set up environment-specific configurations.

6.3 Configuring with profiles

When applications are deployed to different runtime environments, usually some configuration details differ. The details of a database connection, for instance, are likely not the same in a development environment as in a quality assurance environment, and they are different still in a production environment. One way to configure properties uniquely in one environment over another is to use environment variables to specify configuration properties instead of defining them in application.properties and application.yml.

For instance, during development you can lean on the autoconfigured embedded H2 database. But in production, you can set database configuration properties as environment variables like this:

```
% export SPRING_DATASOURCE_URL=jdbc:mysql:/ /localhost/tacocloud
% export SPRING_DATASOURCE_USERNAME=tacouser
% export SPRING_DATASOURCE_PASSWORD=tacopassword
```

Although this will work, it's somewhat cumbersome to specify more than one or two configuration properties as environment variables. Moreover, there's no good way to track changes to environment variables or to easily roll back changes if there's a mistake.

Instead, I prefer to take advantage of Spring profiles. Profiles are a type of conditional configuration where different beans, configuration classes, and configuration properties are applied or ignored based on what profiles are active at run time.

For instance, let's say that for development and debugging purposes, you want to use the embedded H2 database, and you want the logging levels for the Taco Cloud code to be set to `DEBUG`. But in production, you want to use an external MySQL database and set the logging levels to `WARN`. In the development situation, it's easy enough to not set any data source properties and get the autoconfigured H2 database. And as for debug-level logging, you can set the `logging.level.tacos` property for the `tacos` base package to `DEBUG` in application.yml as follows:

```
logging:
  level:
    tacos: DEBUG
```

This is precisely what you need for development purposes. But if you were to deploy this application in a production setting with no further changes to application.yml, you'd still have debug logging for the `tacos` package and an embedded H2 database. What you need is to define a profile with properties suited for production.

3.1 Defining profile-specific properties

One way to define profile-specific properties is to create yet another YAML or properties file containing only the properties for production. The name of the file should follow this convention: application-{profile name}.yml or application-{profile name} .properties. Then you can specify the configuration properties appropriate to that profile. For example, you could create a new file named application-prod.yml that contains the following properties:

```
spring:
  datasource:
    url: jdbc:mysql:/ /localhost/tacocloud
    username: tacouser
    password: tacopassword
logging:
  level:
    tacos: WARN
```

Another way to specify profile-specific properties works only with YAML configuration. It involves placing profile-specific properties alongside nonprofiled properties in application.yml, separated by three hyphens and the `spring.profiles` property to name the profile. When applying the production properties to application.yml in this way, the entire application.yml would look like this:

```
logging:
  level:
    tacos: DEBUG

---
spring:
  profiles: prod

  datasource:
    url: jdbc:mysql:/ /localhost/tacocloud
    username: tacouser
    password: tacopassword

logging:
  level:
    tacos: WARN
```

As you can see, this application.yml file is divided into two sections by a set of triple hyphens (`---`). The second section specifies a value for `spring.profiles`, indicating that the properties that follow apply to the `prod` profile. The first section, on the other hand, doesn't specify a value for `spring.profiles`. Therefore, its properties are common to all profiles or are defaults if the active profile doesn't otherwise have the properties set.

Regardless of which profiles are active when the application runs, the logging level for the `tacos` package will be set to `DEBUG` by the property set in the default profile. But if the profile named `prod` is active, then the `logging.level.tacos` property will be overridden with `WARN`. Likewise, if the `prod` profile is active, then the data source properties will be set to use the external MySQL database.

You can define properties for as many profiles as you need by creating additional YAML or properties files named with the pattern application-{profile name}.yml or application-{profile name}.properties. Or, if you prefer, type three more dashes in application.yml along with another `spring.profiles` property to specify the profile name. Then add all of the profile-specific properties you need. Although there's no benefit to either approach, you might find that putting all profile configurations in a single YAML file works best when the number of properties is small, whereas distinct files for each profile is better when you have a large number of properties.

3.2 Activating profiles

Setting profile-specific properties will do no good unless those profiles are active. But how can you make a profile active? All it takes to make a profile active is to include it in

the list of profile names given to the `spring.profiles.active` property. For example, you could set it in application.yml like this:

```
spring:
  profiles:
    active:
    - prod
```

But that's perhaps the worst possible way to set an active profile. If you set the active profile in application.yml, then that profile becomes the default profile, and you achieve none of the benefits of using profiles to separate the production-specific properties from development properties. Instead, I recommend that you set the active profile(s) with environment variables. On the production environment, you would set `SPRING_PROFILES_ACTIVE` like this:

```
% export SPRING_PROFILES_ACTIVE=prod
```

From then on, any applications deployed to that machine will have the `prod` profile active, and the corresponding configuration properties would take precedence over the properties in the default profile.

If you're running the application as an executable JAR file, you might also set the active profile with a command-line argument like this:

```
% java -jar taco-cloud.jar --spring.profiles.active=prod
```

Note that the `spring.profiles.active` property name contains the plural word *profiles*. This means you can specify more than one active profile. Often, this is with a comma-separated list as when setting it with an environment variable, as shown here:

```
% export SPRING_PROFILES_ACTIVE=prod,audit,ha
```

But in YAML, you'd specify it as a list like this:

```
spring:
  profiles:
    active:
    - prod
```

```
    - audit
    - ha
```

It's also worth noting that if you deploy a Spring application to Cloud Foundry, a profile named `cloud` is automatically activated for you. If Cloud Foundry is your production environment, you'll want to be sure to specify production-specific properties under the `cloud` profile.

As it turns out, profiles aren't useful only for conditionally setting configuration properties in a Spring application. Let's see how to declare beans specific to an active profile.

3.3 Conditionally creating beans with profiles

Sometimes it's useful to provide a unique set of beans for different profiles. Normally, any bean declared in a Java configuration class is created, regardless of which profile is active. But suppose you need some beans to be created only if a certain profile is active. In that case, the `@Profile` annotation can designate beans as being applicable to only a given profile.

For instance, suppose you have a `CommandLineRunner` bean declared in `TacoCloudApplication` that's used to load the embedded database with ingredient data when the application starts. That's great for development but would be unnecessary (and undesirable) in a production application. To prevent the ingredient data from being loaded every time the application starts in a production deployment, you could annotate the `CommandLineRunner` bean method with `@Profile` like this:

```
@Bean
@Profile("dev")
public CommandLineRunner dataLoader(IngredientRepository repo,
    UserRepository userRepo, PasswordEncoder encoder) {

  ...

}
```

Or suppose that you need the `CommandLineRunner` created if either the `dev` profile or `qa` profile is active. In that case, you can list the profiles for which the bean should be created like so:

```
@Bean
@Profile({"dev", "qa"})
public CommandLineRunner dataLoader(IngredientRepository repo,
    UserRepository userRepo, PasswordEncoder encoder) {

  ...

}
```

Now the ingredient data will be loaded only if the `dev` or `qa` profiles are active. That would mean that you'd need to activate the `dev` profile when running the application in the development environment. It would be even more convenient if that `CommandLineRunner` bean were always created unless the `prod` profile is active. In that case, you can apply `@Profile` like this:

```
@Bean
@Profile("!prod")
public CommandLineRunner dataLoader(IngredientRepository repo,
    UserRepository userRepo, PasswordEncoder encoder) {

  ...

}
```

Here, the exclamation mark (`!`) negates the profile name. Effectively, it states that the `CommandLineRunner` bean will be created if the `prod` profile isn't active.

It's also possible to use `@Profile` on an entire `@Configuration`-annotated class. For example, suppose that you were to extract the `CommandLineRunner` bean into a separate configuration class named `DevelopmentConfig`. Then you could annotate `DevelopmentConfig` with `@Profile` as follows:

```
@Profile({"!prod", "!qa"})
@Configuration
public class DevelopmentConfig {

  @Bean
  public CommandLineRunner dataLoader(IngredientRepository repo,
      UserRepository userRepo, PasswordEncoder encoder) {

    ...
```

```
    }
}
```

Here, the `CommandLineRunner` bean (as well as any other beans defined in `DevelopmentConfig`) will be created only if neither the `prod` nor `qa` profile is active.

Summary

- We can annotate Spring beans with `@ConfigurationProperties` to enable injection of values from one of several property sources.
- Configuration properties can be set in command-line arguments, environment variables, JVM system properties, properties files, or YAML files, among other options.
- Use configuration properties to override autoconfiguration settings, including the ability to specify a data source URL and logging levels.
- Spring profiles can be used with property sources to conditionally set configuration properties based on the active profile(s).

Previous chapter
5 Securing Spring

Next chapter
Part 2. Integrated Spring